

Node.js Backend Complete Mastery Notes

Validators · Error Handling · File Uploads · Socket.io
Rate Limiting · Caching · Scaling · Security · Testing

Crafted for Angular developers moving into Full-Stack Node.js
Generated May 2026

TABLE OF CONTENTS

Chapter 1 — Validators with Zod

- Schema patterns
- Middleware integration
- Advanced refinements

Chapter 2 — Global Error Handling

- Error classes
- Async wrapper
- MongoDB error handling

Chapter 3 — File Uploads

- Multer setup
- Cloud storage (Cloudinary/S3)
- Validation

Chapter 4 — Real-time with Socket.io

- Rooms & namespaces
- JWT auth
- Chat rooms

Chapter 5 — Email Service

- Nodemailer setup
- Templates
- Invite flow

Chapter 6 — Rate Limiting & Security

- express-rate-limit
- Helmet
- Sanitization

Chapter 7 — Pagination, Filtering & Sorting

- Query patterns
- Cursor pagination
- Aggregation

Chapter 8 — Caching with Redis

- Setup
- Strategies
- Session/token storage

Chapter 9 — Logging

- Winston
- Log levels
- Production setup

Chapter 10 — Scaling & Performance

- Clustering
- Load balancing
- DB optimization
- PM2

Chapter 11 — Testing

- Jest + Supertest
- Unit tests
- Integration tests

Chapter 12 — API Documentation

- Swagger/OpenAPI
- swagger-jsdoc setup

CHAPTER 1

Validators with Zod

Schema-first input validation for every route

Section 1.1 — Core Patterns

Always validate every incoming request body, query param, and route param. Zod is the best choice for Node.js because it is TypeScript-friendly, composable, and gives excellent error messages out of the box.

Registration Validator

```
// validators/auth.validator.js

import { z } from 'zod';

export const registerValidation = z.object({

  userName : z.string().min(3).max(30)

  .regex(/^[a-zA-Z0-9_]+$/, 'Only letters, numbers & _'),

  email : z.string().email('Invalid email'),

  password : z.string().min(8)

  .regex(/(?=.*[A-Z])/, 'Need 1 uppercase')

  .regex(/(?=.*[0-9])/, 'Need 1 number'),

  confirmPassword: z.string()

}).refine(d => d.password === d.confirmPassword, {

  message : "Passwords don't match",

  path : ['confirmPassword']

});
```

Workspace & Task Validators

```
export const createWorkspaceValidation = z.object({

  name : z.string().min(3).max(50).trim(),

  description : z.string().max(500).optional(),

});

export const addMemberValidation = z.object({
```

```
userId : z.string().uuid(),

role : z.enum(['admin','member','viewer']),

});

export const createTaskValidation = z.object({

title : z.string().min(1).max(100),

description: z.string().optional(),

assignedTo : z.string().uuid().optional(),

dueDate : z.string().datetime().optional(),

priority : z.enum(['low','medium','high']).default('medium'),

status : z.enum(['todo','in-progress','done']).default('todo'),

});
```

Section 1.2 — Reusable Validate Middleware

```
// middlewares/validate.middleware.js

export const validate = (schema) => (req, res, next) => {

const result = schema.safeParse(req.body);

if (!result.success) {

return res.status(400).json({

success : false,

errors : result.error.flatten().fieldErrors

});

}

req.body = result.data; // sanitised & typed data

next();

};

// Usage in routes:

router.post('/register', validate(registerValidation), registerController);

router.post('/workspace', validate(createWorkspaceValidation), createWorkspace);
```

■ **TIP:** Always overwrite `req.body` with `result.data`. Zod strips unknown keys by default, preventing prototype pollution.

Section 1.3 — Query & Param Validation

```
// Validate query strings

export const paginationValidation = z.object({

  page : z.coerce.number().int().min(1).default(1),

  limit : z.coerce.number().int().min(1).max(100).default(10),

  sort : z.enum(['createdAt', 'updatedAt', 'title']).default('createdAt'),

  order : z.enum(['asc', 'desc']).default('desc'),

});


// Middleware for query validation

export const validateQuery = (schema) => (req, res, next) => {

  const result = schema.safeParse(req.query);

  if (!result.success)

    return res.status(400).json({ errors: result.error.flatten() });

  req.query = result.data;

  next();

};
```

CHAPTER 2

Global Error Handling

Centralised, consistent error responses

Section 2.1 — Custom AppError Class

```
// utils/AppError.js

class AppError extends Error {
  constructor(message, statusCode) {
    super(message);

    this.statusCode = statusCode;

    this.status = statusCode >= 400 && statusCode < 500 ? 'fail' : 'error';

    this.isOperational = true; // our own errors

    Error.captureStackTrace(this, this.constructor);
  }
}

export default AppError;

// Usage anywhere in code:

if (!workspace) throw new AppError('Workspace not found', 404);
```

Section 2.2 — Async Wrapper (no more try-catch in every controller)

```
// utils/catchAsync.js

const catchAsync = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

export default catchAsync;

// Controller – clean, no try-catch needed:

export const getWorkspace = catchAsync(async (req, res) => {
  const workspace = await Workspace.findById(req.params.id);
```

```
if (!workspace) throw new AppError('Not found', 404);

res.status(200).json({ success: true, data: workspace });

});
```

Section 2.3 — Global Error Handler Middleware

```
// middleware/errorHandler.js

const handleCastError = e => new AppError(`Invalid ${e.path}: ${e.value}`, 400);

const handleDuplicateKey = e => {

  const field = Object.keys(e.keyValue)[0];

  return new AppError(`${field} already exists`, 409);

};

const handleValidationError = e =>

  new AppError(Object.values(e.errors).map(v=>v.message).join(', '), 400);

const handleJWTError = () => new AppError('Invalid token', 401);

const handleJWTExpired = () => new AppError('Token expired', 401);

export const globalErrorHandler = (err, req, res, next) => {

  err.statusCode = err.statusCode || 500;

  let error = { ...err, message: err.message };

  if (error.name === 'CastError') error = handleCastError(error);

  if (error.code === 11000) error = handleDuplicateKey(error);

  if (error.name === 'ValidationError') error = handleValidationError(error);

  if (error.name === 'JsonWebTokenError') error = handleJWTError();

  if (error.name === 'TokenExpiredError') error = handleJWTExpired();

  res.status(error.statusCode).json({

    success : false,

    message : error.message || 'Internal Server Error',

    ...(process.env.NODE_ENV === 'development' && { stack: err.stack })

  });

};
```

```
});  
  
};
```

■ **TIP:** Register `globalErrorHandler` as the LAST middleware in `app.js`: `app.use(globalErrorHandler)`

CHAPTER 3

File Uploads

Multer + Cloudinary for workspace avatars & task attachments

Section 3.1 — Multer Setup (memory storage for cloud upload)

```
npm install multer cloudinary

// middlewares/upload.middleware.js

import multer from 'multer';

const storage = multer.memoryStorage(); // buffer, not disk

const fileFilter = (req, file, cb) => {

  const allowed = ['image/jpeg', 'image/png', 'image/webp'];

  allowed.includes(file.mimetype) ? cb(null, true)

  : cb(new AppError('Only JPG/PNG/WEBP allowed', 400), false);

};

export const upload = multer({

  storage,

  fileFilter,

  limits: { fileSize: 5 * 1024 * 1024 } // 5 MB

});
```

Section 3.2 — Cloudinary Upload Helper

```
// utils/cloudinary.js

import { v2 as cloudinary } from 'cloudinary';

import streamifier from 'streamifier';

cloudinary.config({

  cloud_name : process.env.CLOUD_NAME,

  api_key : process.env.CLOUD_API_KEY,
```

```
api_secret : process.env.CLOUD_API_SECRET,
});

export const uploadToCloud = (buffer, folder) =>
new Promise((resolve, reject) => {

const stream = cloudinary.uploader.upload_stream(
{ folder, resource_type: 'image' },
(err, result) => err ? reject(err) : resolve(result)
);

streamifier.createReadStream(buffer).pipe(stream);
});

// In your controller:

export const updateAvatar = catchAsync(async (req, res) => {

const result = await uploadToCloud(req.file.buffer, 'workspace/avatars');

await Workspace.findByIdAndUpdate(req.params.id,
{ avatar: result.secure_url });

res.status(200).json({ url: result.secure_url });

});
```

CHAPTER 4

Real-time with Socket.io

Per-project chat rooms + task live updates

Section 4.1 — Server Setup with JWT Auth

```
npm install socket.io

// socket/index.js

import { Server } from 'socket.io';

import jwt from 'jsonwebtoken';

import User from '../models/User.js';

export const initSocket = (httpServer) => {

  const io = new Server(httpServer, {

    cors: { origin: process.env.CLIENT_URL, credentials: true }

  });

  // Auth middleware – runs before any connection

  io.use(async (socket, next) => {

    const token = socket.handshake.auth.token;

    if (!token) return next(new Error('No token'));

    try {

      const decoded = jwt.verify(token, process.env.JWT_SECRET);

      socket.user = await User.findById(decoded.id).lean();

      next();

    } catch { next(new Error('Invalid token')); }

  });

  io.on('connection', (socket) => {

    console.log('Connected:', socket.user.userName);
```

```
// Join project room

socket.on('join:project', (projectId) => {

  socket.join(`project:${projectId}`);

});

// Send message to project room

socket.on('message:send', async ({ projectId, text }) => {

  const msg = await Message.create({

    project: projectId, sender: socket.user._id, text

  });

  io.to(`project:${projectId}`).emit('message:new', {

    ...msg.toObject(), sender: socket.user

  });

});

// Broadcast task updates

socket.on('task:update', ({ projectId, task }) => {

  socket.to(`project:${projectId}`).emit('task:updated', task);

});

socket.on('disconnect', () => {

  console.log('Disconnected:', socket.user.userName);

});

return io;

};
```

■ **TIP:** Attach io to app so controllers can emit events: `app.set('io', io)`. Access with `req.app.get('io')`.

CHAPTER 5

Email Service

Nodemailer + workspace member invitations

Section 5.1 — Nodemailer Setup

```
npm install nodemailer

// utils/email.js

import nodemailer from 'nodemailer';

const transporter = nodemailer.createTransport({

  host : process.env.SMTP_HOST, // smtp.resend.com or smtp.gmail.com

  port : 465,

  secure : true,

  auth : { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS }

});

export const sendEmail = async ({ to, subject, html }) => {

  await transporter.sendMail({

    from: `"WorkSpace App" <${process.env.SMTP_USER}>`,

    to, subject, html

  });

};
```

Section 5.2 — Workspace Invite Flow

```
import crypto from 'crypto';

export const inviteMember = catchAsync(async (req, res) => {

  const { email, role } = req.body;

  const token = crypto.randomBytes(32).toString('hex');

  const expires = Date.now() + 7 * 24 * 60 * 60 * 1000; // 7 days
```

```
await Invite.create({ workspaceId: req.params.id, email, token, role, expires });

const link = `${process.env.CLIENT_URL}/invite/${token}`;

await sendEmail({

  to : email,

  subject : 'You have been invited to a workspace',

  html : `Join the workspace

Accept Invitation`

});

res.status(200).json({ message: 'Invite sent' });

});
```

CHAPTER 6

Rate Limiting & Security

Protect your API from abuse and attacks

Section 6.1 — Rate Limiting

```
npm install express-rate-limit

// middlewares/rateLimiter.js

import rateLimit from 'express-rate-limit';

export const globalLimiter = rateLimit({
  windowMs : 15 * 60 * 1000, // 15 minutes
  max : 100, // requests per window
  message : { error: 'Too many requests, slow down!' },
  standardHeaders: true,
  legacyHeaders : false,
});

export const authLimiter = rateLimit({
  windowMs : 15 * 60 * 1000,
  max : 10, // strict on auth routes
  message : { error: 'Too many login attempts' },
});

// app.js

app.use('/api', globalLimiter);
app.use('/api/auth/login', authLimiter);
app.use('/api/auth/register', authLimiter);
```

Section 6.2 — Helmet (HTTP Security Headers)

```
npm install helmet
```

```
import helmet from 'helmet';

app.use(helmet()); // sets 11 security headers automatically

// What helmet sets for you:

// Content-Security-Policy, X-Frame-Options, X-XSS-Protection

// Strict-Transport-Security (HSTS), X-Content-Type-Options, etc.
```

Section 6.3 — Input Sanitization

```
npm install express-mongo-sanitize hpp

import mongoSanitize from 'express-mongo-sanitize';
import hpp from 'hpp';

app.use(mongoSanitize()); // blocks $gt,$ne injection in body/query

app.use(hpp()); // prevent HTTP parameter pollution

// Also limit request body size:

app.use(express.json({ limit: '10kb' }));
```

■ **TIP:** Security checklist: helmet ✓ mongoSanitize ✓ hpp ✓ rate-limit ✓ body size limit ✓ HTTPS in prod ✓

CHAPTER 7

Pagination, Filtering & Sorting

Handle large datasets efficiently

Section 7.1 — Page-Based Pagination

```
// utils/paginate.js

export const paginate = async (Model, filter, query) => {

  const page = parseInt(query.page) || 1;

  const limit = parseInt(query.limit) || 10;

  const skip = (page - 1) * limit;

  const sort = query.sort || '-createdAt';

  const [data, total] = await Promise.all([

    Model.find(filter).sort(sort).skip(skip).limit(limit).lean(),

    Model.countDocuments(filter)

  ]);

  return {

    data,

    pagination: {

      total, page, limit,

      pages : Math.ceil(total / limit),

      hasNext : page * limit < total,

      hasPrev : page > 1

    }

  };

};

// Usage in controller:

const result = await paginate(Task, { project: req.params.id }, req.query);
```

```
res.json({ success: true, ...result });
```

Section 7.2 — Filtering & Search

```
export const buildTaskFilter = (query, projectId) => {  
  const filter = { project: projectId };  
  
  if (query.status) filter.status = query.status;  
  
  if (query.priority) filter.priority = query.priority;  
  
  if (query.search) filter.title = { $regex: query.search, $options: 'i' };  
  
  if (query.assignee) filter.assignedTo = query.assignee;  
  
  return filter;  
};
```

CHAPTER 8

Caching with Redis

Speed up reads and offload your database

Section 8.1 — Setup & Basic Operations

```
npm install ioredis

// config/redis.js

import Redis from 'ioredis';

const redis = new Redis(process.env.REDIS_URL);

redis.on('connect', () => console.log('Redis connected'));

export default redis;
```

Section 8.2 — Cache-Aside Pattern (most common)

```
// utils/cache.js

import redis from '../config/redis.js';

export const getOrSet = async (key, ttlSec, fetchFn) => {

  const cached = await redis.get(key);

  if (cached) return JSON.parse(cached);

  const data = await fetchFn();

  await redis.setex(key, ttlSec, JSON.stringify(data));

  return data;

};

export const invalidate = (...keys) => redis.del(keys);

// Controller usage:

export const getWorkspaceMembers = catchAsync(async (req, res) => {

  const { id } = req.params;

  const members = await getOrSet(
```

```
`workspace:${id}:members`,

300, // cache for 5 minutes

() => Workspace.findById(id).populate('members').lean()

);

res.json({ success: true, data: members });

});

// When member is added/removed – bust the cache:

await invalidate(`workspace:${workspaceId}:members`);
```

Section 8.3 — Store Refresh Tokens in Redis

```
// Store refresh token with TTL matching token expiry

const REFRESH_TTL = 7 * 24 * 60 * 60; // 7 days in seconds

export const saveRefreshToken = (userId, token) =>

redis.setex(`refresh:${userId}`, REFRESH_TTL, token);

export const getRefreshToken = (userId) =>

redis.get(`refresh:${userId}`);

export const deleteRefreshToken = (userId) =>

redis.del(`refresh:${userId}`);

// On logout – immediately revoke token:

await deleteRefreshToken(req.user.id);
```

■ **TIP:** Redis TTL handles expiry automatically. Never store JWTs in MongoDB — it's slow and you can't invalidate instantly.

CHAPTER 9

Logging with Winston

Replace console.log with structured, levelled logs

Section 9.1 — Winston Setup

```
npm install winston winston-daily-rotate-file

// config/logger.js

import winston from 'winston';

const { combine, timestamp, printf, colorize, errors } = winston.format;

const logFormat = printf(({ level, message, timestamp, stack }) =>
  `${timestamp} [${level}]: ${stack || message}`
);

const logger = winston.createLogger({
  level : process.env.LOG_LEVEL || 'info',
  format : combine(timestamp(), errors({ stack: true }), logFormat),
  transports: [
    new winston.transports.Console({
      format: combine(colorize(), logFormat) // coloured in dev
    }),
    new winston.transports.File({ // error log file
      filename: 'logs/error.log', level: 'error'
    }),
    new winston.transports.File({ // combined log file
      filename: 'logs/combined.log'
    }),
  ]
});
```

```
export default logger;

// Usage – replace console.log everywhere:

logger.info('Server started on port 3000');

logger.warn('Deprecation warning');

logger.error('Database connection failed', err);
```

CHAPTER 10

Scaling & Performance

From single server to production-grade infrastructure

Section 10.1 — Node.js Clustering

Node.js runs on a single thread. Use the cluster module to spawn one process per CPU core, multiplying your server throughput.

```
// cluster.js

import cluster from 'cluster';

import os from 'os';

import app from './app.js';

const CORES = os.cpus().length;

if (cluster.isPrimary) {
  console.log(`Primary ${process.pid} running. Forking ${CORES} workers...`);
  for (let i = 0; i < CORES; i++) cluster.fork();

  cluster.on('exit', (worker) => {
    console.log(`Worker ${worker.process.pid} died. Restarting...`);
    cluster.fork();
  });
} else {
  app.listen(3000, () =>
    console.log(`Worker ${process.pid} listening on 3000`));
}
```

Section 10.2 — PM2 Process Manager (recommended for production)

```
npm install -g pm2

# ecosystem.config.js

export default {
```

```
apps: [{  
  name : 'workspace-api',  
  script : './src/server.js',  
  instances : 'max', // one per CPU  
  exec_mode : 'cluster',  
  watch : false,  
  env_production: {  
    NODE_ENV : 'production',  
    PORT : 3000  
  }  
}]  
};  
  
pm2 start ecosystem.config.js --env production  
  
pm2 save # persist across reboots  
  
pm2 startup # generate startup script  
  
pm2 monit # live dashboard
```

Section 10.3 — MongoDB Performance Optimization

```
// 1. Add indexes on frequently queried fields  
  
TaskSchema.index({ project: 1, status: 1 }); // compound index  
  
TaskSchema.index({ assignedTo: 1 });  
  
MessageSchema.index({ project: 1, createdAt: -1 });  
  
// 2. Use .lean() for read-only queries (returns plain JS objects)  
  
const tasks = await Task.find({ project: id }).lean();  
  
// 3. Select only needed fields  
  
const users = await User.find().select('userName email avatar').lean();  
  
// 4. Aggregation for stats (dashboard counts etc.)
```

```
const stats = await Task.aggregate([  
  
  { $match: { project: mongoose.Types.ObjectId(projectId) } },  
  
  { $group: { _id: '$status', count: { $sum: 1 } } }  
  
]);
```

Section 10.4 — Horizontal Scaling Concepts

Load Balancer: Distributes traffic across multiple server instances (Nginx, AWS ALB).

Stateless Servers: Store sessions/tokens in Redis, not in-memory. Any server can handle any request.

Database Replica Sets: MongoDB replica set gives you automatic failover + read scaling.

CDN for Static Assets: Serve images/files from Cloudinary or AWS CloudFront — never from your app server.

Message Queues: Use BullMQ + Redis for heavy tasks (email sending, report generation) to avoid blocking requests.

■ **TIP:** Key rule for scaling: make your app stateless. If it works with 1 server, it'll work with 100.

CHAPTER 11

Testing with Jest & Supertest

Confidence that your API works correctly

Section 11.1 — Setup

```
npm install -D jest supertest @jest/globals
```

```
// package.json
```

```
"scripts": { "test": "node --experimental-vm-modules node_modules/.bin/jest" },
```

```
"jest": { "testEnvironment": "node", "extensionsToTreatAsEsm": [".js"] }
```

Section 11.2 — Integration Test Example

```
// tests/auth.test.js
```

```
import request from 'supertest';
```

```
import app from '../src/app.js';
```

```
import mongoose from 'mongoose';
```

```
beforeAll(async () => {
```

```
  await mongoose.connect(process.env.MONGO_TEST_URI);
```

```
});
```

```
afterAll(async () => {
```

```
  await mongoose.connection.dropDatabase();
```

```
  await mongoose.disconnect();
```

```
});
```

```
describe('POST /api/auth/register', () => {
```

```
  it('should register a user', async () => {
```

```
    const res = await request(app).post('/api/auth/register').send({
```

```
      userName: 'testuser', email: 'test@test.com',
```

```
      password: 'Password1', confirmPassword: 'Password1'
```

```
});

expect(res.status).toBe(201);

expect(res.body.data).toHaveProperty('token');

});

it('should reject duplicate email', async () => {

  await request(app).post('/api/auth/register').send({

    userName: 'user2', email: 'test@test.com',

    password: 'Password1', confirmPassword: 'Password1'

  });

  const res = await request(app).post('/api/auth/register').send({

    userName: 'user3', email: 'test@test.com',

    password: 'Password1', confirmPassword: 'Password1'

  });

  expect(res.status).toBe(409);

});

});
```

CHAPTER 12

API Documentation with Swagger

Auto-generated, interactive API docs

Section 12.1 — Setup

```
npm install swagger-jsdoc swagger-ui-express

// config/swagger.js

import swaggerJSDoc from 'swagger-jsdoc';

import swaggerUi from 'swagger-ui-express';

const options = {

  definition: {

    openapi: '3.0.0',

    info: { title: 'Workspace API', version: '1.0.0' },

    components: {

      securitySchemes: {

        bearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT' }

      }

    },

    security: [{ bearerAuth: [] }]

  },

  apis: ['./src/routes/*.js'],

};

export const swaggerSpec = swaggerJSDoc(options);

export const swaggerMiddleware = swaggerUi.serve;

export const swaggerHandler = swaggerUi.setup(swaggerSpec);

// app.js

app.use('/api/docs', swaggerMiddleware, swaggerHandler);
```

```
// Docs live at: http://localhost:3000/api/docs
```

Section 12.2 — Documenting a Route with JSDoc

```
/**
 * @swagger
 * /api/auth/login:
 * post:
 *   summary: Login user
 *   tags: [Auth]
 *   requestBody:
 *     required: true
 *     content:
 *       application/json:
 *         schema:
 *           type: object
 *           required: [email, password]
 *           properties:
 *             email: { type: string, example: user@mail.com }
 *             password: { type: string, example: Password1 }
 *           responses:
 *             200:
 *               description: Login successful
 *             401:
 *               description: Invalid credentials
 */
router.post('/login', validate(loginValidation), loginController);
```

Learning Roadmap Summary

Validators	Error Handling	File Uploads	Socket.io	Email
Rate Limiting	Redis Cache	Logging	Scaling	Testing

You have built a solid workspace application. Master these chapters in order, apply each concept to your own project, and you will have production-level Node.js skills. Keep building! ■